

Lab 04: Factor analysis 1

Author: **N.J. de Winter** (*n.j.de.winter@vu.nl*)

Assitant Professor Vrije Universiteit Amsterdam

Statistics and Data Analysis Course

Modified after: Trauth, Martin H., et al. MATLAB recipes for earth sciences. Vol. 34. Berlin: Springer, 2007.

Learning goals:

- Apply and improve your knowledge of Python and Jupyter
- Get familiar with factor analysis
 - Learn to run a Principle Component Analysis (PCA) in Python
 - Understand and apply tools to assess whether the *variables* in a dataset can be reduced
 - Interpret *scores* and *loadings* and *eigenvalues* of a PCA result
- Develop a feeling for how statistical tools can help you, but you still require *your interpretation* to draw conclusions.

Introduction

In this lab, we will start working with **Factor analysis**. More specifically, we will apply a technique called **Principle Component Analysis** to a dataset to test whether the information in the data can be *reduced* by finding a small set of *components* which replace the *variables* in the dataset while containing (almost) the same information.

This all sounds quite technical, and that is because it is. **Factor analysis can be a hard concept to wrap your head around**, and it is normal to be confused about it in the beginning. However, it is also a very powerful tool for data reduction, which plays an important role in modern science and in our society as a whole and is used for a surprising number of things!

This is why we spend two labs practicing it. Do not despair, we will go through this step by step so you will have plenty of time to practice.

We will start by loading the packages we will need. You know how to do this by now, so we will not give you the code any more from now on.

Exercise 1: Load the `numpy` package and the `pyplot` package (part of `matplotlib`) and `import` the function `PCA` from the `decomposition` package which is part of `sklearn`. Don't forget to add the statement you used before to allow plots to be visualized in Jupyter

```
In [ ]: # Make sure our figures show up in Jupyter

# The 'numpy' package contains some handy functions
# The 'matplotlib' package contains tools needed to plot our data and results
# Load the PCA function from the decomposition package in sklearn
```

In this lab, we are going to use the same dataset as in *Lab 03*. You know how to load this data by now (if not, have a sneak peak at the previous lab). Don't forget that if you are using Spyder, you need to first define your working directory, and that if you are using Jupyter, you need to move the dataset into the same folder as your notebook.

Exercise 2: Load *Lab03.txt* in Python and explore the data to familiarize yourself again with its structure.

```
In [ ]: # Load data
# inspect data
```

If a `print()` of the dataset confuses you, recall again type of data is in this dataset. You should be able to answer the questions below easily if you do.

Question 1: What do the *observations* in this dataset represent?

Answer 1:

Question 2: What do the *variables* in this dataset represent?

Answer 2:

Question 3: What type of information does each datapoint in the dataset contain? What is the unit of this data? What is the minimum and maximum value a datapoint can have?

Answer 3:

Before we dive into factor analysis, it is very important that we recap the structure of the data and explore the correlations between variables we can already observe before doing "fancy" statistics. The easiest way to do this is to create a correlation matrix and to visualize the correlations between the variables in some way. Luckily, you have already learned how to do that in the previous Lab, so you can recycle the code from **Lab03**.

Exercise 3: Calculate the correlation matrix for the dataset and plot the correlations using a heatmap (use the `imshow` function)

```
In [ ]: # Create a correlation matrix of the mineral content

# Create a vector of variable names (minerals)

# Flip the correlation matrix for plotting:

# Plot the correlation matrix with colors representing the degree of correlation:

# Add a title to the graph

# Add the mineral labels:

# Display the colorbar as a legend:
```

Question 4: Which (groups of) minerals are highly correlated to each other?

Answer 4:

This is a typical example of a dataset in which the variables contain a lot of overlap in terms of information. The fact that some variables are so highly correlated shows us that they teach us (almost) the same thing about our samples. From a statistical and data science point of view, this is very **inefficient**! We can likely **summarize the information contained in this dataset using a small number of more smartly chosen variables**. This is the goal of factor analysis. Let's get started!

We will use the `PCA` function to perform a Principle Component Analysis on our dataset.

Exercise 4: First, have a look at the `PCA` function using the `help()` function

```
In [ ]:
```

OK, that is a lot of information. Don't worry, you don't need to understand all the options you have with this function. Sometimes, it is easier to look at the `help()` for a function online. The `help` -generated webpage for a function contains the same information as you have just exported using the `help()` function, but it is nicely formatted and easier to read. [This is the help page for the `PCA` function](#)

What you need to keep in mind is that the syntax (the way you call the function in Python) is a bit different from what you are used to so far. It works as follows: First you create an object that contains the settings you want for your PCA. Then you `fit` that object on your data to get PCA results. We'll go through it step by step below:

```
In [ ]: pca = PCA() # We first call the PCA function to create our PCA object
```

Note that we did not specify any settings for our PCA, which means we are using the default settings. You can find out what these are in the `help()` documentation.

```
In [ ]: scores = pca.fit_transform(data) # Now we apply the new pca object to our data to get our PCA scores
print(scores) # Examine the scores
```

The *scores* give you the values for each *principle component* for each sample. Remember that the *principle components* can be regarded as the "new variables". The *loadings* of the PCA are stored in the variable `pca.components_`:

```
In [ ]: print(pca.components_)
```

In order to obtain a matrix in which columns represent the principal component *loadings* in descending order of explained variability, we need to transpose the `pca.components_` variable:

```
In [ ]: loadings = np.transpose(pca.components_) # Create a new object with Loadings in columns ordered from highest to Least amount of explained variance.
print(loadings) # Examine the result
```

Question 5: What type of information can you get from the loadings of your PCA? How can you interpret them?

Answer 5:

Let's plot the loadings of the first principal component. In the code below, note that we use the first column of the `loadings` matrix (`loadings[:, 0]`) to isolate the loadings for the first principle component.

```
In [ ]: a = np.arange(1,10) # Create a vector with numbers for the variables
plt.figure(2)
plt.plot(a, loadings[:, 0], 'o', clip_on = False) # Plot scatter plot of Loadings for each variable.
plt.plot(a, np.zeros(np.size(a)), color = 'red') # Plot a horizontal red line indicating Loading of zero
for i, label in enumerate(minerals): # Loop through the variable names (minerals)
    plt.text(a[i] + 0.2, loadings[:, 0][i], label, fontsize = 8) # Plot the names of the variables next to the points
plt.xlim([1, 9]) # Set the Limits for the horizontal axis
plt.ylim([-1, 1]) # Set the Limits for the vertical axis
plt.title('PC 1') # Set the plot title
```

Question 6: Which variables load highly on the first principal component?

Answer 6:

Question 7: Knowing what you do about this dataset (refer back to **Lab03** if you don't remember!), what might principle component 1 represent?

Answer 7:

Exercise 5: Now repeat the process for Principle Component 2 and make your interpretation like you did for the first one by answering **Question 8** and **Question 9** below.

```
In [ ]:
```

Question 8: Which variables load strongly on principle component 2?

Answer 8:

Question 9: What might principle component 2 represent?

Answer 9:

Now we will make a cross plot of the loadings of the first two principle components:

```
In [ ]: plt.figure(4)
plt.scatter(loadings[:, 0], loadings[:, 1]) # Plot Loadings of PC1 vs PC2
plt.xlim([-0.6, 0.8]) # Set horizontal axis Limits
plt.ylim([-0.6, 0.8]) # Set vertical axis Limits
plt.axhline(color = 'r') # Create red horizontal line for y = 0
plt.axvline(color = 'r') # Create red vertical line for x = 0
for i, label in enumerate(minerals):
    plt.text(loadings[:, 0][i] + 0.02, loadings[:, 1][i], label, fontsize = 8) # Label the variables
plt.xlabel('PC1 loadings') # Provide Label for horizontal axis
plt.ylabel('PC2 loadings') # Provide Label for vertical axis
```

Question 10: What can we learn from this scatterplot? Does the result surprise you after what you have learned in Lab03 and from your correlation matrix?

Answer 10:

We can also make a scatter plot of the *scores* of the samples (observations) for PC1 and PC2 instead of the *loadings*. The `scores` variable gives the principal component *scores* for all samples.

Exercise 6: Try to make this scores scatterplot yourself for PC1 vs PC2.

Tip: You can recycle much of the code used for the crossplot above, but be careful with the labels you use for the plot and the limits of your axes.

In []:

Question 11: What does this plot tell you about the different samples, especially if you compare it to the loadings crossplot?

Answer 11:

So far, we have only looked at the first two principle components, but there are more. In fact, a PCA always initially yields a number of components equal to the number of variables in the dataset. However, the amount of variance in the dataset that is explained by a component decreases down the list. To keep track of how much variance each component explains (and therefore how important it is), we can calculate the *eigenvalues* of the components. We do this by dividing the variance explained by each component by the total amount of variance in the dataset, and we multiply by 100% to make the numbers easier to interpret:

In []:

```
percent_explained = 100 * pca.explained_variance_ / np.sum(pca.explained_variance_)
print(percent_explained)
```

To make this result easy to interpret, we can plot these eigenvalues in a scatterplot. The code below should be quite familiar to you now.

In []:

```
a = np.arange(1, 10) # Create a vector with numbers for the components
plt.figure(6)
plt.plot(a, percent_explained, 'o', clip_on = False) # Plot scatter plot of eigenvalues for each component.
plt.xlim([1, 9]) # Set the Limits for the horizontal axis
plt.ylim([0, 100]) # Set the Limits for the vertical axis
plt.xlabel('PCA component') # Provide Label for horizontal axis
plt.ylabel('Percent of variance explained') # Provide Label for vertical axis
plt.title('Eigenvalues per component') # Set the plot title
```

Since the entire goal of factor analysis is to **reduce** the data, we want to get rid of those principle components that explain (almost) no variance and only keep the components that are important enough.

Question 12: Where would you draw the line? How many components would you keep? And why?

Answer 12: